

MÓDULO 5 — INTEGRAÇÃO FULL STACK

OrbitBoard

Painel didático de gestão de projetos, tarefas e equipe — aplicação full stack usada como estudo de caso da capacitação em IA e Transformação Digital.

EQUIPE

Thalia Rodrigues Pinto

Pamela do Nascimento Bernardo

Thiago Serra Andrade Leite

Objetivo

Aplicar, na prática, os conteúdos de integração full stack: revisar e executar uma aplicação com front-end, back-end/API e infraestrutura containerizada, validar a comunicação HTTP/JSON, ajustar CORS e variáveis de ambiente, documentar o contrato da API e registrar evidências de testes.

Compreender a arquitetura full stack e o papel de cada camada

Executar front-end, back-end/API e infraestrutura containerizada

Validar a comunicação HTTP/JSON entre front-end e API

Documentar o contrato da API e registrar evidências

Arquitetura resumida

1

Front-end

React 18 + Vite, servido em produção por Nginx.

porta 8080

2

Back-end

API REST em .NET 8 / ASP.NET Core, com dados mantidos em memória.

porta 5200

3

Infraestrutura

Containers Docker para front-end e back-end, orquestrados por docker-compose.yml.

Docker Compose

Detalhamento completo em docs/arquitetura.md

Stack utilizada

Camada	Tecnologias
Front-end	React 18, Vite, React Router, Nginx (produção)
Back-end	.NET 8, ASP.NET Core Web API, Swashbuckle (Swagger)
Infraestrutura	Docker, Docker Compose

Como executar — Docker Compose

TERMINAL

```
$ cp .env.example .env
```

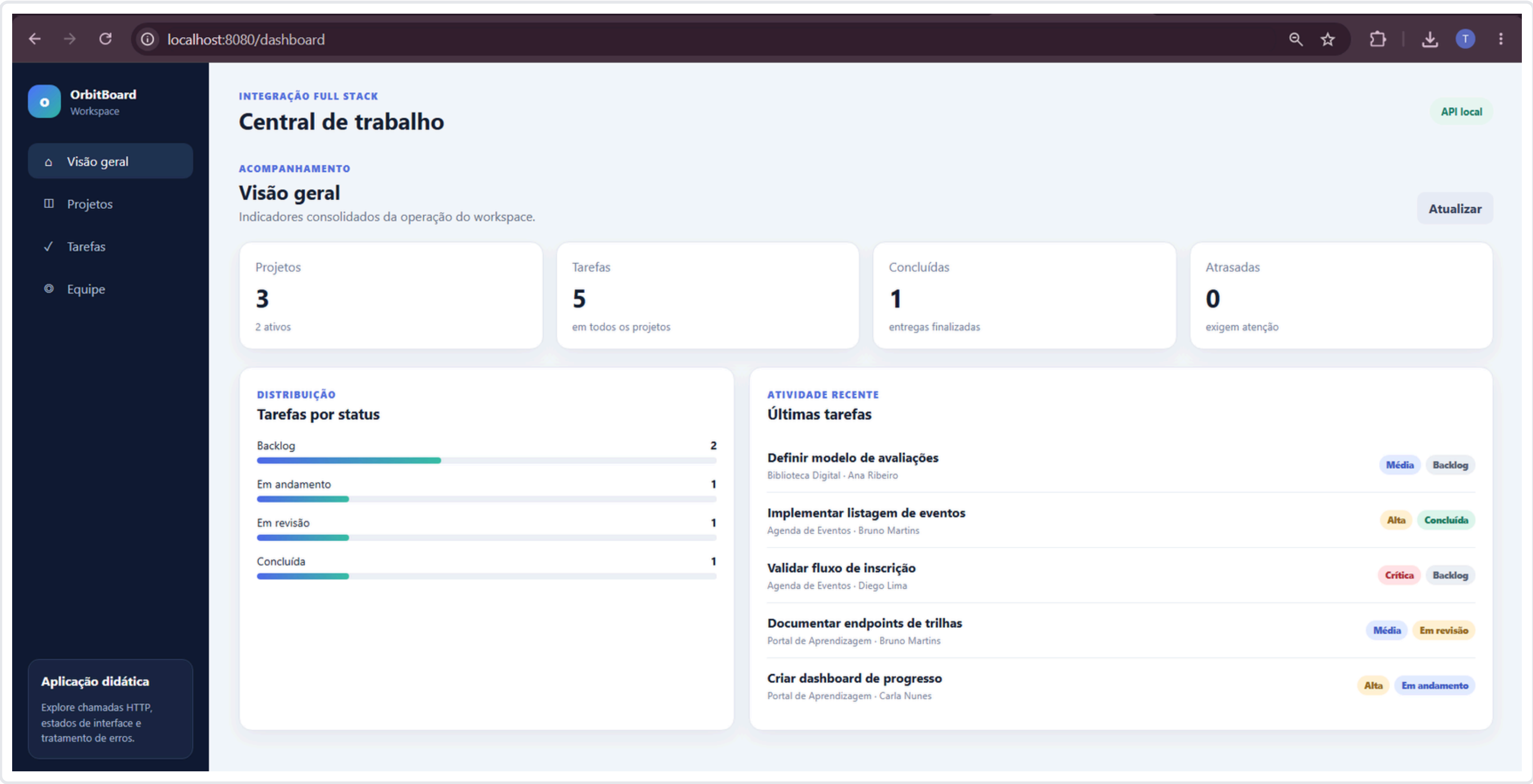
```
$ docker compose up --build
```

O Docker isola a execução em containers, evitando problemas de compatibilidade entre arquiteturas.

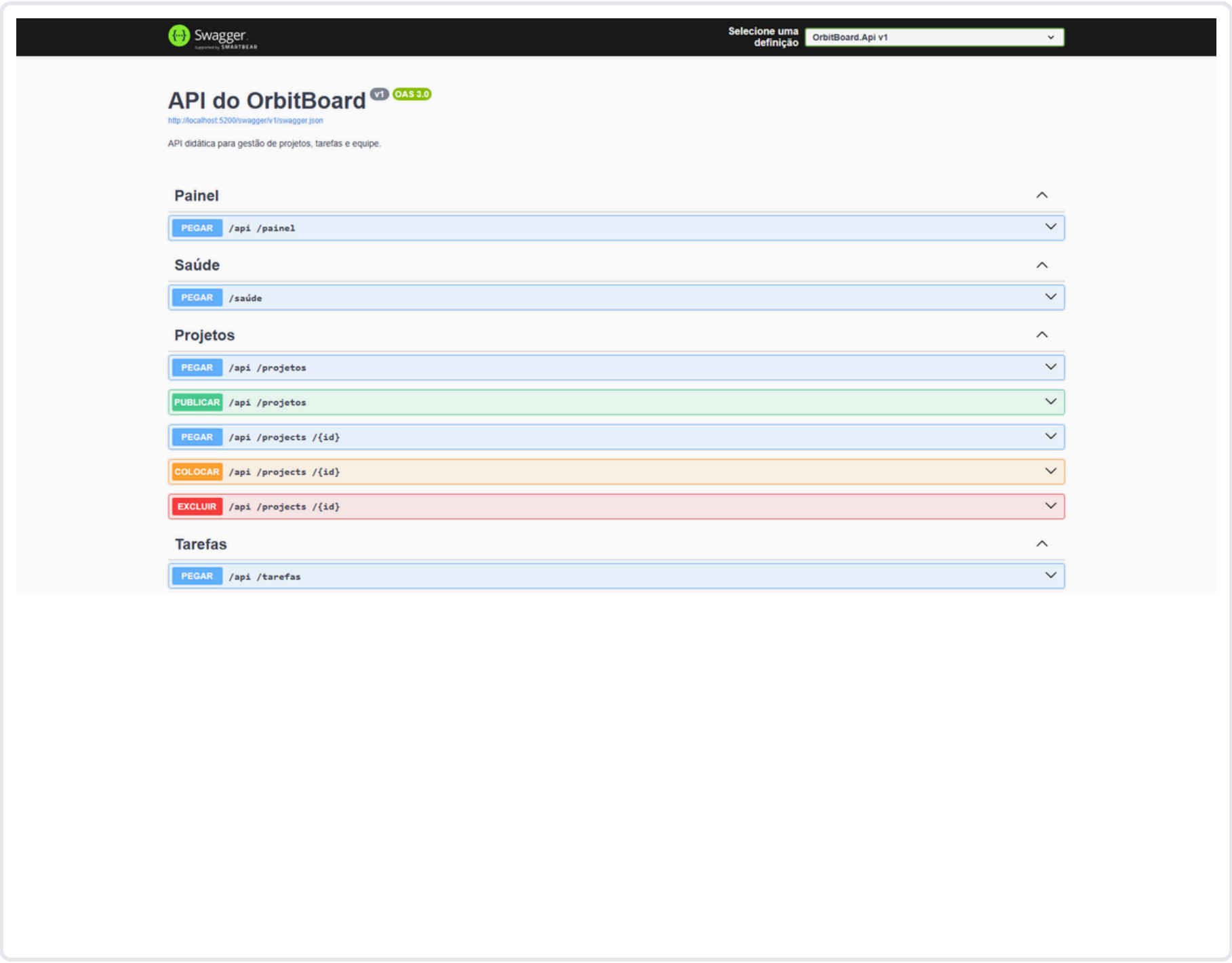
ENDEREÇOS DE ACESSO

- **Front-end** <http://localhost:8080>
- **Back-end / API** <http://localhost:5200>
- **Swagger** <http://localhost:5200/swagger>
- **Health API** <http://localhost:5200/health>
- **Health Front-end** <http://localhost:8080/health>

Demonstração — Dashboard



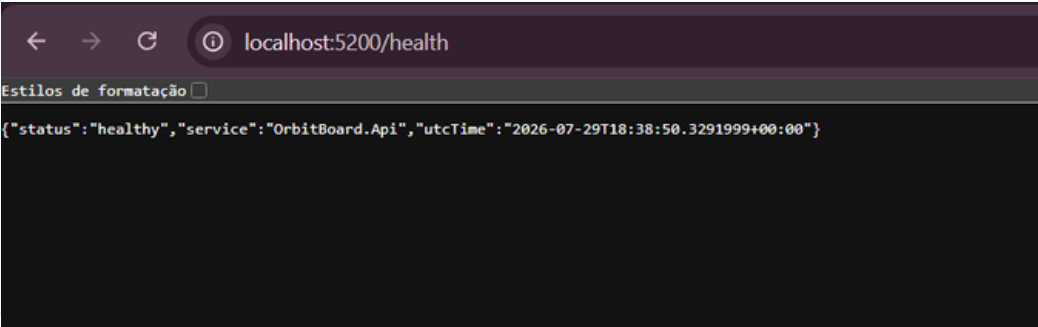
API REST & Swagger



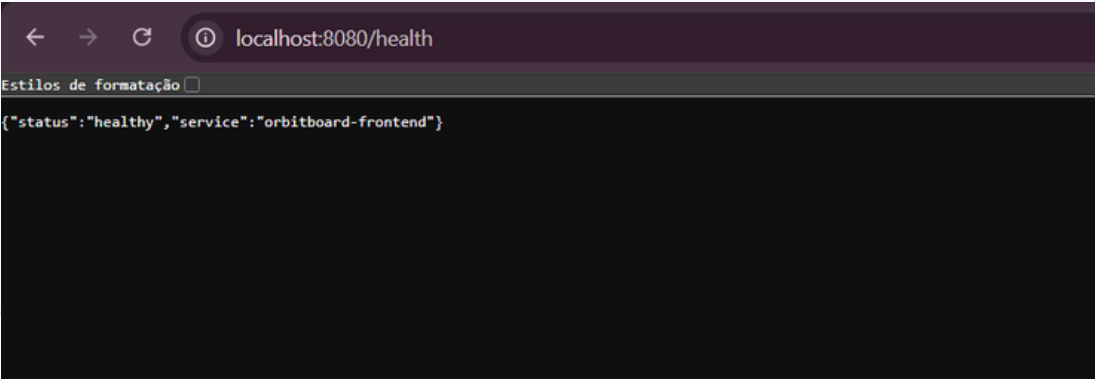
Método	Endpoint	Descrição
GET	/api/dashboard	Métricas e tarefas
GET/POST	/api/projects	Listar / criar
GET/PUT/DEL	/api/projects/{id}	CRUD projeto
GET/POST	/api/tasks	Listar / criar tarefas
PATCH	/api/tasks/{id}/status	Alterar status
GET	/health	Health check

Testes e evidências

HEALTH — API



HEALTH — FRONT-END



STATUS GERAL

● Front-end

healthy

● Back-end API

healthy

● Build Docker

5/5 up

EXECUÇÃO — docker compose up --build

```
✓ Container orbit-board-equipe-08-frontend-1 Created
2.2s
Attaching to backend-1, frontend-1
frontend-1 | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
frontend-1 | /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
frontend-1 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
frontend-1 | 10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
frontend-1 | 10-listen-on-ipv6-by-default.sh: info: /etc/nginx/conf.d/default.conf differs from the packaged version
frontend-1 | /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
frontend-1 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
frontend-1 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
frontend-1 | /docker-entrypoint.sh: Configuration complete; ready for start up
frontend-1 | 2026/07/29 18:34:07 [notice] 1#1: using the "epoll" event method
```

Contribuição de cada integrante

TS

Thiago Serra Andrade Leite

Aplicação-base

Utilizou o projeto-modelo do professor para montar a aplicação (Dockerfile, docker-compose, ci.yml inicial) e repassou o repositório à equipe.

Pd

Pamela do Nascimento Bernardo

CI/CD

Testou e ajustou o pipeline de integração contínua (triggers do workflow) até o CI funcionar corretamente.

TR

Thalia Rodrigues Pinto

Documentação & Apresentação

Executou a aplicação via Docker Compose, coletou as evidências de funcionamento (prints) e preparou os slides desta apresentação.

Obrigado!

github.com/ev1lauronBR/orbit-board-equipe-08